

数据结构与算法课程设计

动态规划：300.最长递增子序列

1、程序代码

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        if (nums.size() <= 1)
            return nums.size();
        vector<int> dp(nums.size(), 1);
        int res = 0;

        for (int i = 1; i < nums.size(); i++) {
            for (int j = 0; j < i; j++) {
                if (nums[i] > nums[j])
                    dp[i] = max(dp[i], dp[j] + 1);
            }
            if (dp[i] > res)
                res = dp[i];
        }
        return res;
    }
};

int main() {
    int t;
    cin >> t;
    while (t--) {
        Solution sol;
        vector<int> nums;
        int n;
        cin >> n;
        for (int i = 0; i < n; ++i) {
            int num;
            cin >> num;
            nums.push_back(num);
        }
        int lisLength = sol.lengthOfLIS(nums);
        cout << "最长递增子序列的长度是: " << lisLength << endl;
    }
    return 0;
}
```

2、运行结果

```
3
8
10 9 2 5 3 7 101 18
最长递增子序列的长度是: 4
6
0 1 0 3 2 3
最长递增子序列的长度是: 4
7
7 7 7 7 7 7 7
最长递增子序列的长度是: 1

Process returned 0 (0x0)    execution time : 4.521 s
Press any key to continue.
```

3、实习总结与体会

学到了动态规划 (dp)

(1) 动态规划：状态转移公式（递推公式）很重要，但动规不仅仅只有递推公式，更重要的还是思考。

- 动态规划模板：

1. 确定dp数组（dp table）以及下标的含义
2. 确定递推公式
3. dp数组如何初始化
4. 确定遍历顺序
5. 举例推导dp数组

(2) 动态规划题debug：把dp数组打印出来，看看究竟是不是按照自己思路推导的

(3) 最长递增子序列：

- 如果数组长度小于等于1，直接返回数组长度，即本身。
- 初始化一个动态规划数组 `dp`，长度与输入数组相同，所有元素初始化为1。每个元素本身可以看作长度为1的递增子序列。
- 遍历数组，从第二个元素开始（索引 `i = 1`）。对于每个元素 `nums[i]`，再遍历其之前的所有元素（索引 `j` 从0到 `i-1`）。
- 如果 `nums[i] > nums[j]`，说明 `nums[i]` 可以接在 `nums[j]` 后面形成一个更长的递增子序列。
- 此时，更新 `dp[i]` 为 `max(dp[i], dp[j] + 1)`。